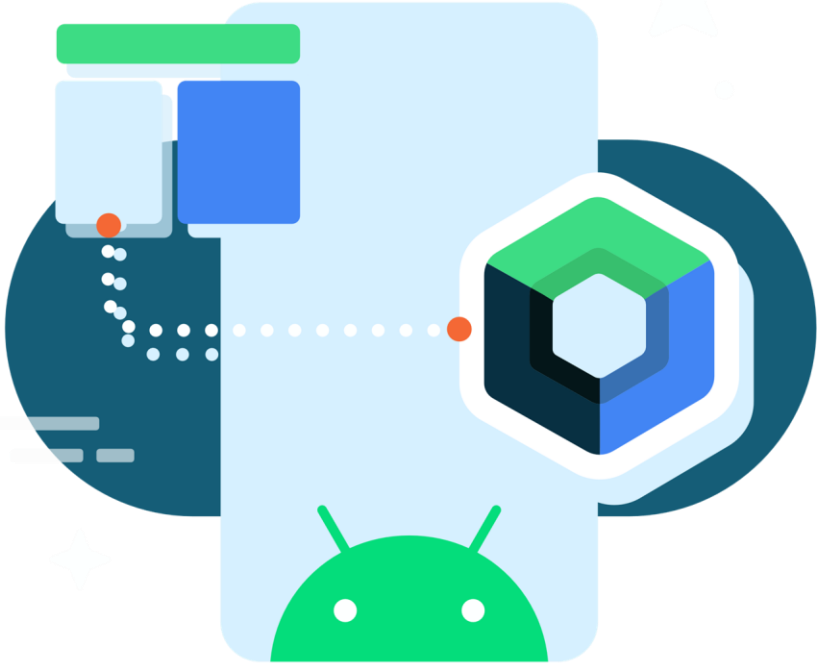
رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

Mobile Application Development

State Management in Jetpack Compose



State in Compose

What is State?

State: is any value that can change over time, affecting UI display.

Compose handles state **declaratively**:

UI is automatically updated when the state changes.

No manual UI updates required (unlike XML or imperative approaches).

Why is this different?

In **imperative** UI:

- Explicit commands to update the UI (**setText**, **setVisibility**).

In **declarative** UI (Compose):

- UI is a **function of state**.
- **Recomposition** happens automatically

Using remember and mutableStateOf

Concept:

- Compose tracks state using **mutableStateOf**.
- Wrap the state variable with **remember** to survive recompositions.

```
@Composable
fun Counter() {
  var count by remember { mutableStateOf(0) }

  Column {
    Text("Count: $count")
    Button(onClick = { count++ }) {
      Text("Increment")
    }
  }
}
```

- Changing the value triggers recomposition automatically.

Watching Recomposition in Action

Example Explained:

- Every time the `count` is incremented:
  - The UI (`Text`) recomposes with the updated count.

Visual Representation:

```
State Change (count++) → Recomposition → UI Update
```

Note:

- Compose optimizes recomposition to only update what's necessary.

When and Why to Use rememberSaveable

- Regularly `remember` doesn't survive configuration changes (screen rotation).
- Use `rememberSaveable` to retain UI state across configuration changes.

Example:

```
@Composable
fun ShowTextField() {
    var text by rememberSaveable { mutableStateOf("") }

    TextField(
        value = text,
        onValueChange = { newValue ->
            text = newValue
        },
        label = {
            Text("Email")
        }
    )
}
```

- After rotation, text persists.

Stateless vs Stateful Composables

Stateful Composable

- Holds and manages its own state.
- Less reusable, tightly coupled with specific logic.

Stateless Composable

- Doesn’t hold state itself - receives state from parameters **(preferred)**.
- Highly reusable and testable.

Best Practice:

- Prefer **stateless** composables.
- "Lift state up" (**state hoisting**) to achieve reusability.

State Hoisting (Lifting State Up)

Concept:

- Moving state management to a higher composable (parent) to make child composables stateless.

Example:

Before (**Stateful**):

```
@Composable
fun StatefulCounter() {
    var count by remember { mutableStateOf(0) }

    Button(onClick = { count++ }) {
        Text("Count: $count")
    }
}
```

After (**Stateless** with **Hoisting**):

```
@Composable
fun StatelessCounter(
    count: Int,
    onIncrement: () -> Unit
) {
    Button(onClick = onIncrement) {
        Text("Count: $count")
    }
}

// Usage (parent composable):
@Composable
fun StatefulCounter() {
    var count by remember { mutableStateOf(0) }
    StatelessCounter(count) { count++ }
}
```

Using derivedStateOf

- Optimize recomposition when the **new state depends on the existing state (another state)**.
- Creates derived state efficiently.

Example:

```
val count by remember { mutableStateOf(0) }
val isEven by derivedStateOf { count % 2 == 0 }

Text("Number is even? $isEven")
```

- Only recomposes when the derived value (`isEven`) actually changes.

Best Practices for Stateful Components

- **Lift the state up** whenever possible.
- Keep composables **small**, **focused**, and **reusable**.
- Use **state hoisting** and **callbacks** effectively.
- Minimize unnecessary recomposition using:
  - `derivedStateOf`
  - Smart usage of state variables

UI Interactions & State Management

Examples of interactions that affect the state:

- Button → `onClick`
- TextField → `onValueChange`
- Switch, CheckBox state changes

Interaction Example:

- Each interaction updates the state
  updates the state → recomposition
  recomposition → UI updated automatically.

```
@Composable
fun ShowCheckBox(modifier: Modifier = Modifier) {
    var checked by remember { mutableStateOf( value: false) }

    Checkbox(
        checked = checked,
        onCheckedChange = { newState ->
            checked = newState
        },
        modifier = modifier
    )
}
```

Passing State & Callbacks Between Components

- Child composables receive state and callbacks through parameters.

Example:

```
@Composable
fun Parent() {
  var name by remember { mutableStateOf("") }

  Child(name = name, onNameChange = { name = it })
}

@Composable
fun Child(name: String, onNameChange: (String) -> Unit) {
  TextField(value = name, onValueChange = onNameChange)
}
```

- Child component remains stateless and reusable.

Intro to StateFlow (ViewModel Integration Preview)

- `StateFlow`: Kotlin Flow optimized for state management.
- **Emits** updates that UI can observe reactively.

Why use StateFlow?

- Reactive and ~~lifecycle-aware.~~
- Ideal for ViewModel ↔ UI communication

Quick Example:

ViewModel:

```
class CounterViewModel: ViewModel() {

    private val _count = MutableStateFlow( value: 0)
    val count = _count.asStateFlow()

    fun increment() {
        _count.value++
    }
}
```

Compose UI:

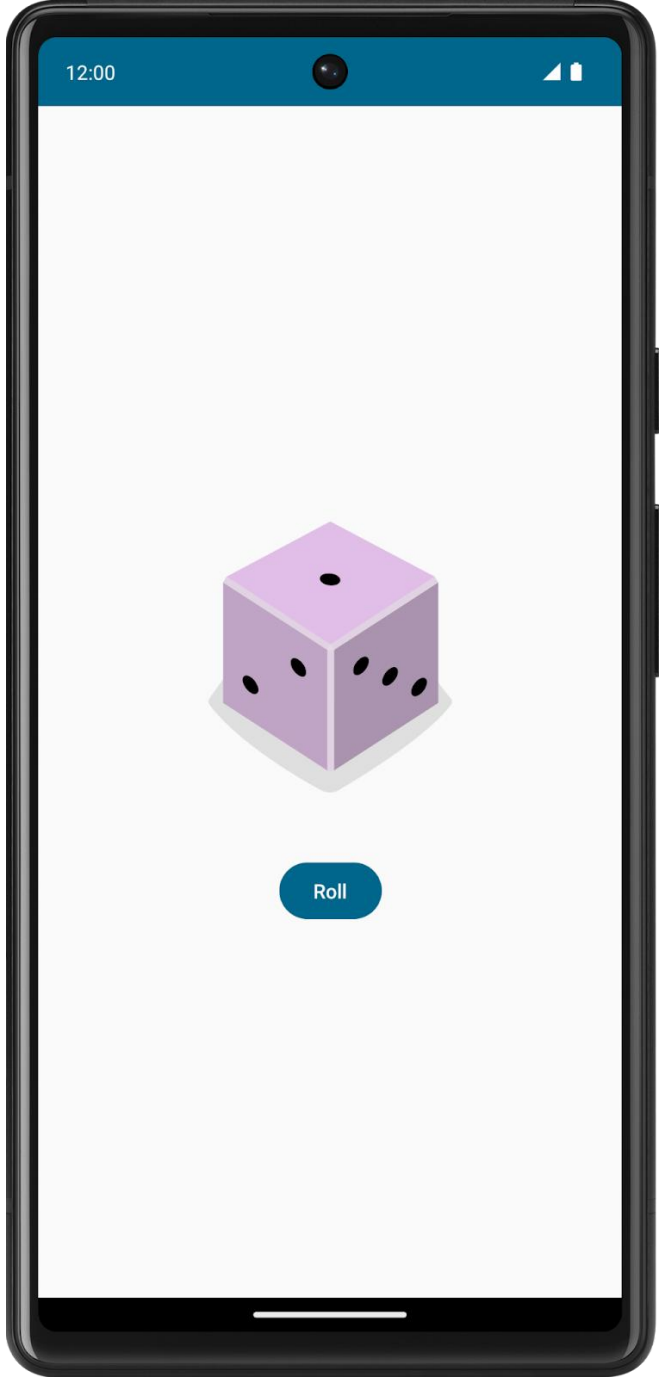
```
@Composable
fun CounterScreen(
    viewModel: CounterViewModel,
    modifier: Modifier = Modifier
) {
    val count by viewModel.count.collectAsStateWithLifecycle()

    Button(
        onClick = { viewModel.increment() },
        modifier = modifier,
    ) {
        Text( text: "Count: $count")
    }
}
```

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

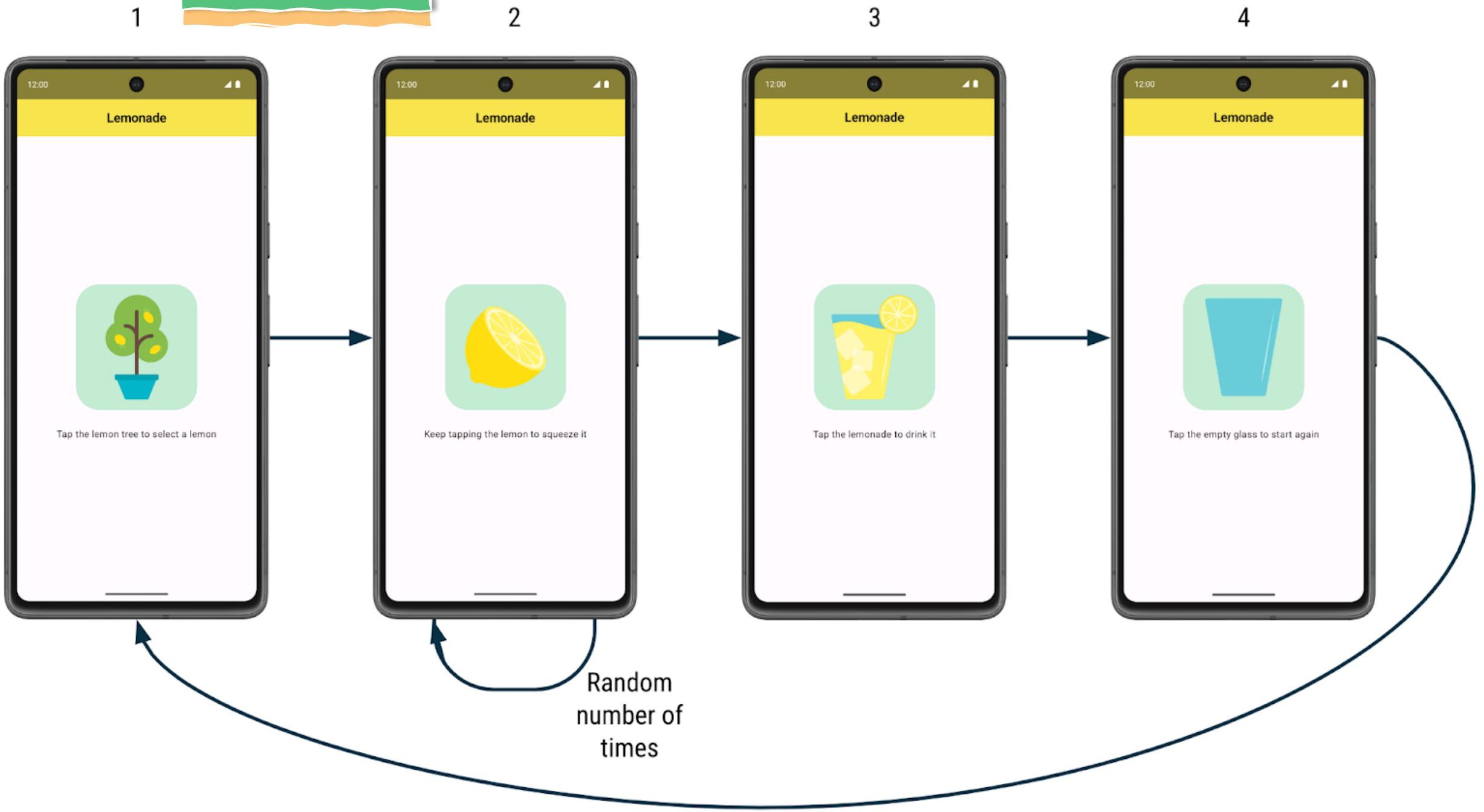
Exercise 1

Use Jetpack Compose with Kotlin to build an interactive **Dice Roller** app that lets users tap a Button composable to roll a dice. The outcome of the roll is shown with an Image composable on the screen.



https://developer.android.com/codelabs/basic-android-kotlin-compose-build-a-dice-roller-app#0

Exercise 2


1
2
3
4
12:00
Lemonade
Tap the lemon tree to select a lemon
Keep tapping the lemon to squeeze it
Tap the lemonade to drink it
Tap the empty glass to start again
Random number of times

Quiz Time

Key Takeaways

- Compose manages state **declaratively**.
- Use `remember` and `mutableStateOf` for local UI state.
- Persist state across configuration with `rememberSaveable`.
- Keep composables **stateless** by **lifting state up**.
- Use `derivedStateOf` to improve performance.
- `StateFlow` sets the stage for **robust MVVM architecture**.

رواد مصر الرقمية

وزارة الاتصالات
وتكنولوجيا المعلومات

THANK YOU